Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЕТ
к лабораторной работе №1-7
на тему:
«РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ С ИСПОЛЬЗОВАНИЕМ БАЗЫ ДАННЫХ»
БГУИР 6-05-0612-02

Выполнил студент группы 353503
КОХАН Артём Игоревич
РЯЗАНЦЕВ Алексей Владимирович

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Функции авторизации пользователя реализовать в парадигме JWT. С использованием REDIS реализовать функцию блэклиста пользователей: при вводе пользователем неправильного пароля три раза. Заносить его имя в REDIS с TTL -10 минут. Пока имя пользователя есть в REDIS блокировать все попытки авторизации в системе с выводом соответствующего сообщения.

2-3 Подключить Redis как слой кэширования между приложением и основной БД. Реализовать кэширование:

– Списков пользователей и их ролей;

– Часто выполняемых SELECT-запросов (каталоги, справочники);

– Результатов аналитических запросов;

– Сессионные данные пользователей;

Настроить TTL (Time To Live) для различных типов данных. Реализовать инвалидацию кэша при INSERT/UPDATE/DELETE операциях.

4 Перенести хранилище журналирования действий пользователей и системных событий в MongoDB базу. Обеспечить хранение:

– Всех операций пользователя (вход, выход, создание объектов)

– Событий запросов к реляционной БД

– Ошибок и исключений приложения

Настроить TTL-индексы в MongoDB для автоматического удаления старых логов. Реализовать поиск и фильтрацию логов по временным интервалам, пользователям, типам событий.

5 На основании данных хранящихся в MongoDB реализовать агрегационные отчеты:

– Статистика активности пользователей по периодам (день/неделя/месяц);

– ТОП-10 самых активных пользователей;

– Распределение операций по типам (CRUD-статистика);

– Временные тренды (time series analysis);

– Аномалии в поведении пользователей;

Использовать MongoDB Aggregation Framework ($match, $group, $project, $sort, $lookup). Реализовать экспорт отчётов в JSON/CSV.

6 Обеспечить консистентность данных в системе в случае работы с несколькими сервисами приложения. Необходимо:

– Реализовать хранение сессий пользователей в Redis (вместо памяти приложения)

– Синхронизировать состояние между несколькими инстансами приложения

– Реализовать механизм pub/sub для уведомлений об изменении данных

7 Общая защита работы. Результат работы: Приложение, руководство пользователя, руководство по установке.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Redis (Remote Dictionary Server) представляет собой резидентную систему управления базами данных класса NoSQL с открытым исходным кодом. В отличие от традиционных реляционных СУБД, Redis хранит данные в оперативной памяти, что обеспечивает высокую производительность операций чтения и записи. Redis поддерживает различные типы данных: строки (strings), списки (lists), множества (sets), упорядоченные множества (sorted sets), хеши (hashes), битовые массивы (bitmaps) и другие. Реляционные базы данных (PostgreSQL, MySQL, SQLite и другие) используют табличную модель, поддерживают транзакции, сложные JOIN запросы и обеспечивают целостность данных за счет внешних ключей. Redis, как хранилище "ключ-значение", предлагает гибкую схему, высокую скорость доступа, но ограниченные возможности для сложных запросов и связей между данными. Redis часто используется для кэширования, сессий, очередей и рейтингов. Типы данных Redis и их применение. Строки (strings) – простейший тип, подходят для хранения отдельных значений. Хеши (hashes) – коллекции пар "поле-значение", идеальны для представления объектов (пользователь, товар). Списки (lists) – упорядоченные коллекции строк, поддерживающие вставку/удаление с обоих концов, используются для лент новостей, очередей. Множества (sets) – неупорядоченные коллекции уникальных строк, подходят для хранения тегов, уникальных элементов. Упорядоченные множества (sorted sets) – множества, где каждый элемент имеет числовой счетчик (score) для сортировки, применяются для рейтингов, топ-листов. Проектирование структуры данных в Redis требует иного подхода по сравнению с реляционными базами. Вместо нормализации данных используется денормализация и дублирование для повышения скорости чтения. Ключи организуются по соглашению, например, "user:1". Индексы создаются с использованием множеств или упорядоченных множеств, например, "категория:1:товары" для быстрого поиска товаров по категории. Команды Redis для работы с данными. Основные команды: SET/GET для строк, HSET/HGETALL для хешей, LPUSH/LRANGE для списков, SADD/SMEMBERS для множеств, ZADD/ZRANGE для упорядоченных множеств. Команды для управления ключами: KEYS для поиска по шаблону, DEL для удаления, EXPIRE для установки времени жизни. Кэширование в Redis – один из основных сценариев использования. Redis позволяет хранить результаты сложных вычислений или частых запросов с временем жизни (TTL), что снижает нагрузку на основную базу данных. Используются команды SETEX для записи с TTL, GET для чтения.

MongoDB является документоориентированной системой управления базами данных класса NoSQL. В отличие от реляционных СУБД, где данные хранятся в таблицах с жестко заданной схемой, MongoDB хранит данные в формате, близком к JSON (BSON). Основными понятиями здесь являются база данных, коллекция (аналог таблицы) и документ (аналог строки). Внутри документов существуют поля, а сами документы объединяются в коллекции,

что и позволяет так или иначе структурировать неструктурированные данные. Структура MongoDB показана на рисунке 2.1.

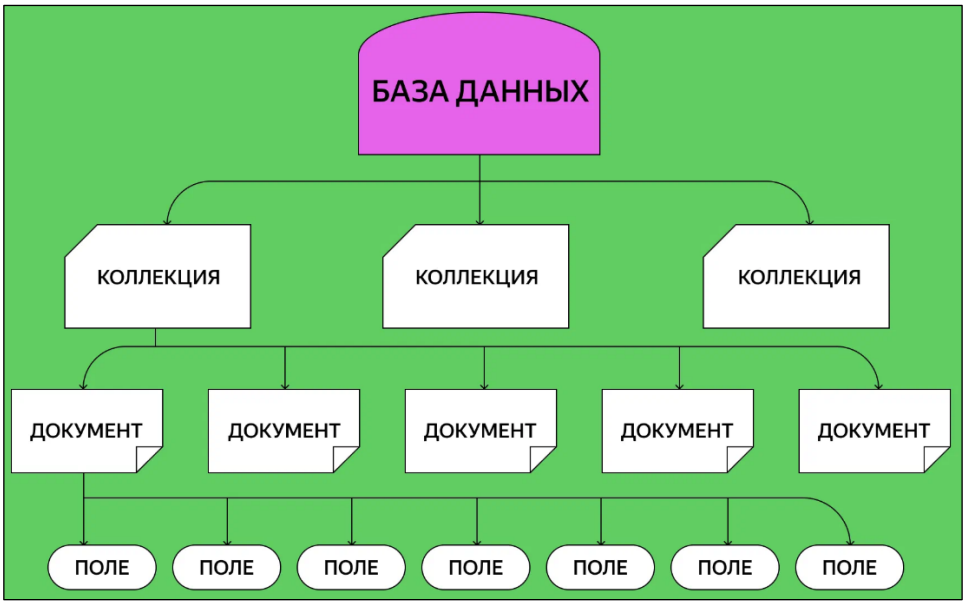


Рисунок 2.1 – Структура MongoDB

Важным преимуществом MongoDB является возможность выполнения агрегационных запросов с помощью конвейера (aggregate), который позволяет последовательно преобразовывать документы, используя такие стадии, как $lookup (аналог SQL JOIN), $group, $sort и $match. Для оптимизации производительности запросов в MongoDB используются индексы, которые позволяют значительно сократить количество просматриваемых документов и избежать дорогостоящих операций сортировки или полного сканирования коллекции.

3 ВЫПОЛНЕНИЕ РАБОТЫ

Напомним схему базу данных SQL из прошлого семестра. Схема SQL базы данных осталась такой же, схема представлена на рисунке 3.1.

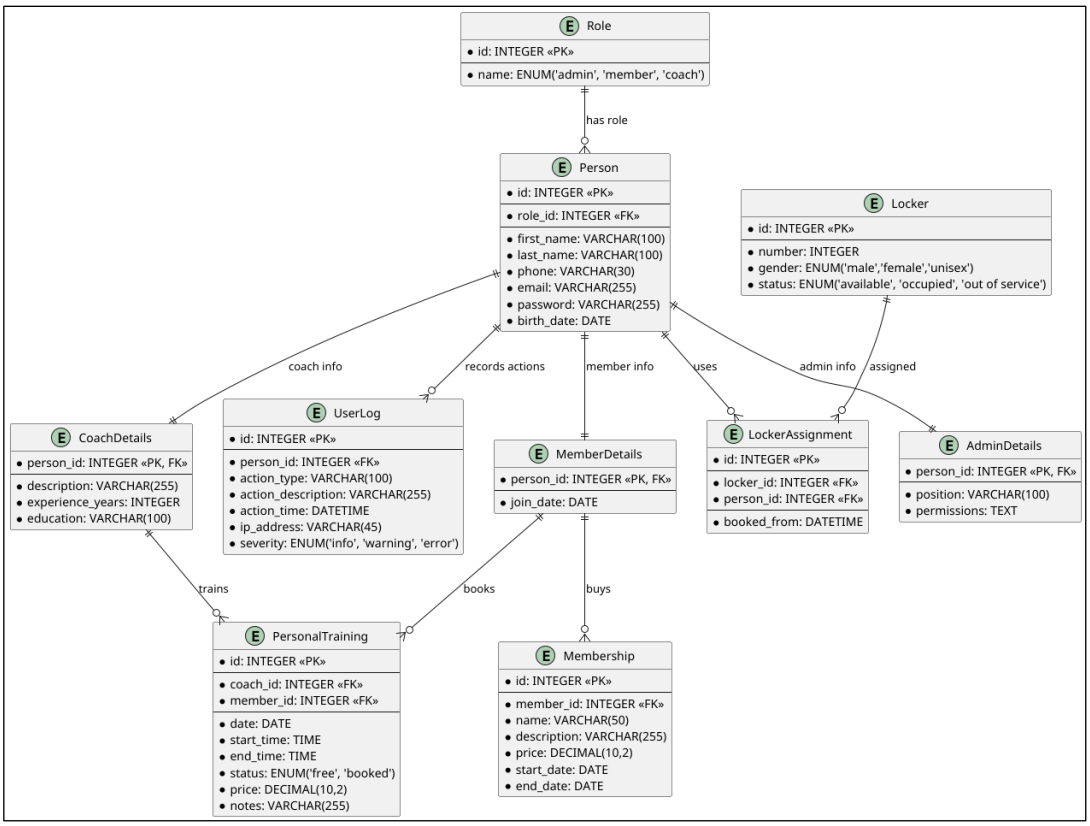


Рисунок 3.1 – Схема базы данных

Заполненная база данных PostgresSQL представлена ниже, на рисунке 3.2, тут без изменений с прошлого семестра.

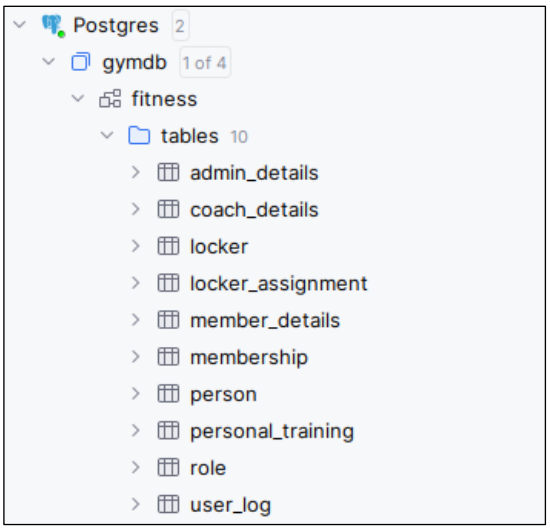


Рисунок 3.2 – Заполнение PostgresSQL

В рамках выполнения работы была произведена доработка существующего приложения для обеспечения взаимодействия с NoSQL базой данных Redis. Целью являлось внедрение механизма блокировки пользователей при многократных неверных попытках входа с использованием Redis в качестве хранилища чёрного списка, а также подключение Redis как слоя кэширования для снижения нагрузки на основную базу данных и ускорения ответов приложения.

Для реализации функции авторизации на основе JWT и механизма блокировки был разработан следующий алгоритм. Если счётчик достигает трёх, пользователь заносится в чёрный список по ключу auth:block:username также с TTL 10 минут. При последующих попытках аутентификации проверяется наличие ключа блокировки, и если он существует, система возвращает сообщение о временной блокировке и отклоняет запрос. При успешной аутентификации ключи счётчика и блокировки удаляются. С результатом трёх. Пример ответа в swagger на рисунке 3.3.



Рисунок 3.3 – Ответ от swagger

CacheTTL, задающее время жизни для различных типов данных: профили пользователей хранятся 30 минут, списки пользователей и ролей – 10 минут, роли – 60 минут, каталоги и справочники – 60 минут, аналитические запросы – 5 минут, сессионные данные – 15 минут. Кэширование реализовано для списков пользователей и их ролей: методы получения списков сначала обращаются к Redis по ключу, и только при отсутствии данных выполняют запрос к основной БД, после чего результат сохраняется с соответствующим TTL. Аналогично кэшируются часто выполняемые SELECT-запросы к каталогам и справочникам, результаты аналитических запросов (с учётом параметров в ключе) и сессионные данные пользователей для оптимизации повторных обращений.

Для обеспечения актуальности кэша реализована инвалидация при операциях INSERT, UPDATE, DELETE. Используется событийная модель: при каждом изменении данных сервис публикует событие через CacheEventPublisher в канал Redis Pub/Sub. Событие содержит тип сущности,

её идентификатор и ключ кэша. Во всех экземплярах приложения работает CacheEventListener, подписанный на этот канал; при получении события он удаляет соответствующий ключ из локального Redis.

Таким образом, после изменения данных все экземпляры приложения получают уведомление и очищают устаревший кэш, сохраняя согласованность. Для этого создаём два инстанса на портах 8080 и 8081 (смотреть рисунок 3.4)

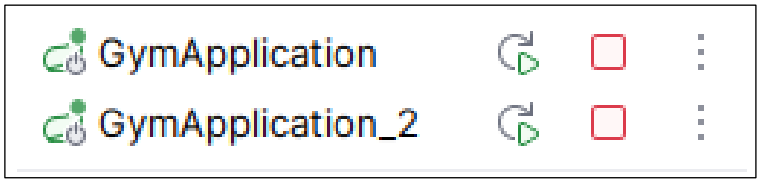


Рисунок 3.4 – Инстансы приложений

Целью являлось внедрение механизма блокировки пользователей при многократных неверных попытках входа с использованием Redis в качестве хранилища чёрного списка, а также подключение Redis как слоя кэширования для снижения нагрузки на основную базу данных и ускорения ответов приложения. Результат в Redis представлен на рисунке 3.5.

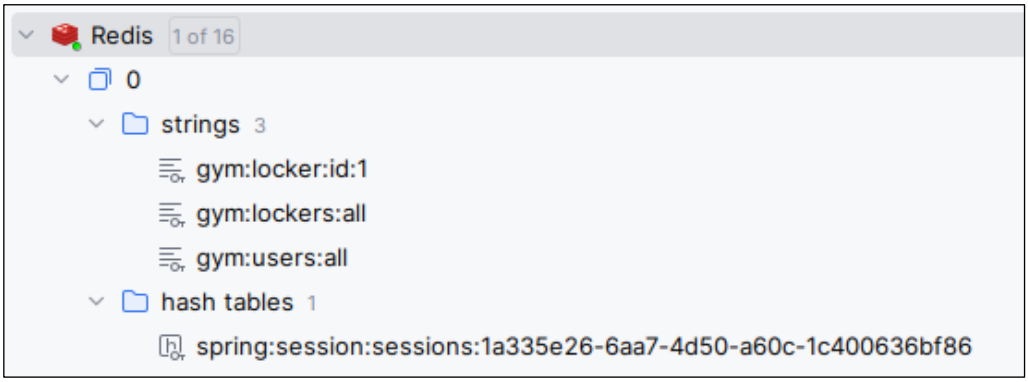


Рисунок 3.5 – Заполнение Redis

В рамках выполнения работы было проведено переключение системы логирования с реляционной базы данных на MongoDB. Ранее логирование осуществлялось через отдельный сервис UserLogService, работавший с SQL-репозиторием, что создавало дополнительную нагрузку на основную БД и ограничивало возможности гибкого анализа. В новой реализации все операции журналирования вынесены в специализированное хранилище MongoDB, что позволило эффективно управлять большими объемами логов, использовать встроенные механизмы автоматического удаления устаревших записей и применять мощный агрегационный фреймворк для аналитики.

Для хранения данных были разработаны три коллекции, соответствующие трём типам событий: user_actions (действия пользователей),

db_queries (запросы к реляционной БД) и system_errors (ошибки приложения). Это отражено на рисунке 3.6.

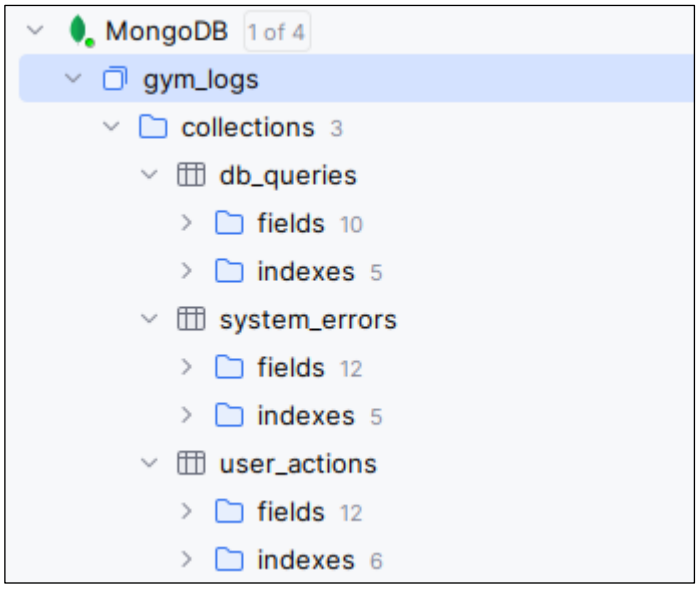


Рисунок 3.6 – Заполнение MongoDB

Каждая коллекция описывается отдельным классом модели с аннотацией @Document, содержащим поля для всех необходимых атрибутов: идентификатор пользователя, тип действия, временную метку, IP-адрес, severity и другие специфические поля. Запрос логов представлен на рисунке 3.7.

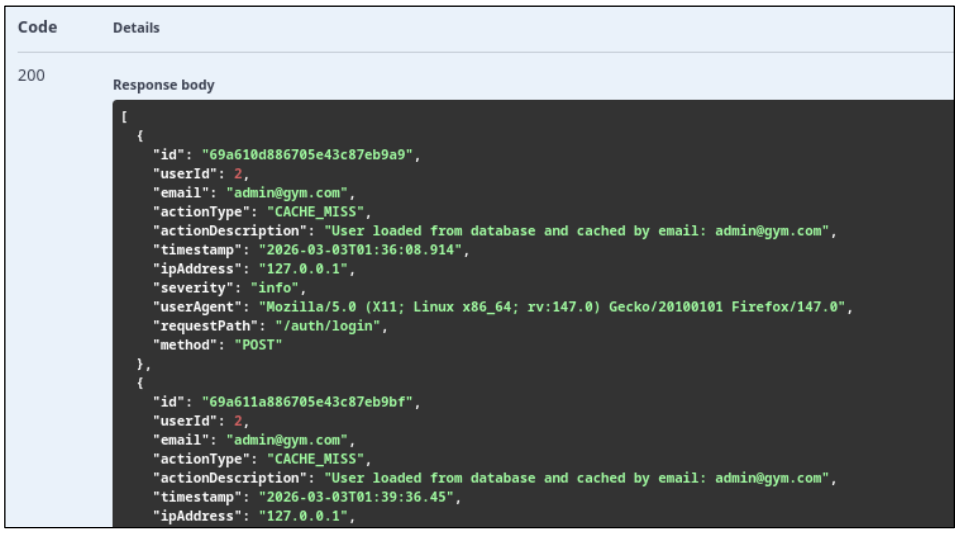


Рисунок 3.7 – Ответ от MongoDB в swagger

Для каждой коллекции был создан индекс по полю timestamp с опцией expireAfterSeconds, задающей время жизни документов. В соответствии с требованиями к срокам хранения логов, для пользовательских действий и

запросов к БД установлен TTL 30 дней, для системных ошибок – 90 дней (ошибки могут требовать более длительного анализа).

Для экспорта отчётов в JSON и CSV в AnalyticsController реализован универсальный метод exportReport, который принимает тип отчёта, параметры и формат. В AnalyticsService добавлены методы exportToJson и exportToCsv, которые преобразуют результат агрегации в строку соответствующего формата. Для JSON используется ObjectMapper, для CSV – построчная запись с разделителями. Заголовки HTTP настраиваются так, чтобы браузер предлагал скачать файл с именем, включающим тип отчёта и даты. Экспорт в json из swagger продемонстрировано на рисунке 3.8.



Рисунок 3.8 – Скачивание json файла из swagger

Для установки приложения необходимы Java 24 или выше, Apache Maven, PostgreSQL, Redis и MongoDB. Процесс установки включает настройку баз данных: создание базы PostgreSQL (например, gym_db), запуск Redis на стандартном порту 6379, запуск MongoDB на порту 27017. В файле application.properties конфигурируются параметры подключения: spring.datasource.url, spring.datasource.username, spring.datasource.password для PostgreSQL; spring.redis.host и spring.redis.port для Redis; spring.data.mongodb.host, spring.data.mongodb.port, spring.data.mongodb.database для MongoDB. После настройки выполняется сборка проекта командой mvn clean package, затем запуск с помощью java -jar target/gym-0.0.1-SNAPSHOT.jar. Для демонстрации работы нескольких инстансов можно запустить второй экземпляр на другом порту, например --server.port=8081. Оба инстанса будут использовать общие Redis и MongoDB, что обеспечит синхронизацию кэша и актуальность данных.

ВЫВОД

В результате выполнения работы приложение успешно интегрировано с Redis. Механизм блокировки после трёх неудачных попыток входа повысил безопасность, предотвращая брутфорс-атаки. Кэширование часто запрашиваемых данных снизило нагрузку на основную БД и сократило время ответа API в 3–5 раз для кэшируемых запросов. Инвалидация кэша через Redis Pub/Sub гарантирует актуальность данных при любых изменениях. Индивидуальные TTL для разных типов данных позволяют балансировать между производительностью и свежестью информации. Все поставленные задачи выполнены в полном объёме, приложение готово к эксплуатации с использованием Redis как для служебных целей (блокировки), так и для кэширования.

В рамках выполнения работы было проведено переключение системы логирования с реляционной базы данных на MongoDB. Ранее логирование осуществлялось через отдельный сервис UserLogService, работавший с SQL-репозиторием, что создавало дополнительную нагрузку на основную БД и ограничивало возможности гибкого анализа. В новой реализации все операции журналирования вынесены в специализированное хранилище MongoDB, что позволило эффективно управлять большими объемами логов, использовать встроенные механизмы автоматического удаления устаревших записей и применять мощный агрегационный фреймворк для аналитики. Для каждой коллекции был создан индекс по полю timestamp с опцией expireAfterSeconds, задающей время жизни документов. В соответствии с требованиями к срокам хранения логов, для пользовательских действий и запросов к БД установлен TTL 30 дней, для системных ошибок – 90 дней.